

1.3 Expressions

Sam once spent a summer working as a cashier in a popular fast-food outlet. One of his responsibilities was to make change for customers when they paid with cash. While he enjoyed the mental exercise of doing the math in his head, he immediately started wondering how this could best be done with the computer. After a few iterations, he came up with a program to make light work of his most tedious task...

Objectives

By the end of this class, you will be able to:

- Represent simple equations in C++.
- Understand the differences between integer division and floating point division.
- See how to use the modulus operator to solve math and logic problems.

Prerequisites

Before reading this chapter, please make sure you are able to:

- Choose the best data-type to represent your data (chapter 1.2).
- Declare a variable (chapter 1.2).
- Display text and numbers on the screen (chapter 1.1).

Overview

Computer programs perform mathematical operations much the way one would expect. There are a few differences, however, owing to the way computers store numbers. For example, there is no distinction between integers and floating point numbers in Algebra. This means that dividing one by two will yield a half. However, in C++, integers can't store the number 0.5 or $\frac{1}{2}$. Also, a variable can update its value in C++ where in Algebra it remains constant through the entire equation. These challenges along with a few others makes performing math with C++ a little tricky.

In C++, mathematical equations are called **expressions**. An expression is a collection of values and operations that, when evaluated, result in a single value.

Evaluating Expressions

As you may recall from our earlier discussion of how computers work, a CPU can only perform elementary mathematical operations and these can only be done one at a time. This means that the compiler must break complex equations into simple ones for them to be evaluated correctly by the CPU. To perform this task, things are done in the following order:

1. Variables are replaced with the values they contain
2. The order of operations are honored: parentheses first and assignment last
3. When there is an integer being compared/computed with a float, it is converted to a float just before evaluation.

Step 1 - Variables are replaced with values

Every variable refers to a location of memory. This memory location is guaranteed to be filled with 1's and 0's. In other words, there is *always* a value in a variable and that value can always be accessed at any time. Sometimes the value is meaningless. Consider the following example:

```
{
    int number;
    cout << number << endl;           // the output is different every time because
                                      //   the variable number was never initialized
}
```

Since the variable was never initialized, the value is not predictable. In other words, whoever last used that particular location in memory left data lying around. This means that there is some random collection of 1's and 0's in that location. We call this state uninitialized because the programmer never got around to assigning a value to the variable `number`. All this could be rectified with a simple:

```
int number = 0;
```

The first step in the expression evaluation process is to substitute the variables in the expression with the values contained therein. Consider the following code:

```
{
    int ageHumanYears = 4;
    int ageDogYears = ageHumanYears * 7;
}
```

In this example, the first step of evaluating the last statement is to substitute `ageHumanYears` with 4.

```
int ageDogYears = 4 * 7;
```

Step 2 - Order of Operations

The order of operations for mathematical operators in C++ is:

Operator	Description
()	Parentheses
++ --	Increment, Decrement
* / %	Multiply, Divide, Modulo
+ -	Addition, Subtraction
= += -= *= /= %=	Assign, Add-on, Subtract-from, Multiply onto, Divide from, Modulo from.

This should be very familiar; it is similar to the order of operations for Algebra. There are, of course a few differences

Increment ++

Because it is possible to change the value of a variable in C++, we have an operator designed specifically for the task. Consider the following code:

```
{
    int age = 10;
    age++;
    cout << age << endl;        // the output is 11
}
```

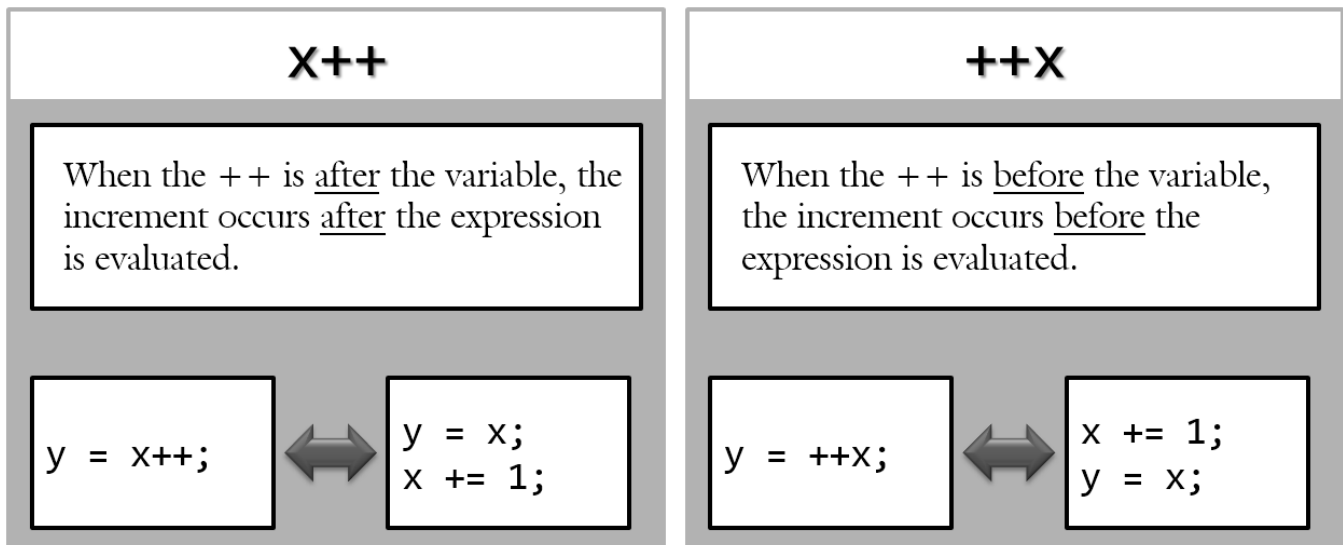
In this example, the `age++` statement serves to add one to the current value of `age`. Of course, `age--` works in the opposite way. There are two flavors of the increment (and decrement of course) operators: increment before the expression is evaluated and increment after. To illustrate, consider the following example:

```
{
    int age = 10;
    cout << age++ << endl;      // the output is 10 and the new value of age is 11
}
```

In this example, we increment the value of `age` *after* the expression is evaluated (as indicated by the `age++` rather than `++age` where we would evaluate *before*). Therefore, the output would be 10 although the value of `age` would be 11 at the end of execution. This would not be true with:

```
{
    int age = 10;
    cout << ++age << endl;      // the output is 11 and the new value of age is 11
}
```

In this case, `age` is incremented *before* the expression is evaluated and the output would be 11. In short:



Multiplication *

In C++ (and most other computer languages for that matter), the multiplication operator is an asterisk *. You cannot use the dot operator (ex: .), the multiplication x (ex: ×), or put a number next to a variable (ex: 7y) as you can in standard algebra notation.

```
{
    float answer1 = 1.2 * 2.3;    // the value of answer1 is 2.76
    int    answer2 = 2 * 3;        // the value of answer2 is 6
}
```

Division /

Floating point division (/) behaves the way it does in mathematics. Integer division, on the other hand, does not. The evaluation of integer division is always an integer. In each case, the remainder is thrown away. To illustrate this, consider the following:

```
{
    int    answer1 = 19 / 10;
    float  answer2 = 19.0 / 10.0;
    cout << answer1 << endl;        // the output is 1
        << answer2 << endl;        // the output is 1.9
}
```

In this case, the output of the first line is not 1.9 because the variable `answer1` cannot store a floating point value. When 19 is divided by 10, the result is 1 with a remainder of 9. Therefore, `answer1` will get the value 1 and the remainder is discarded. To get 1.9, we need to use floating point division.

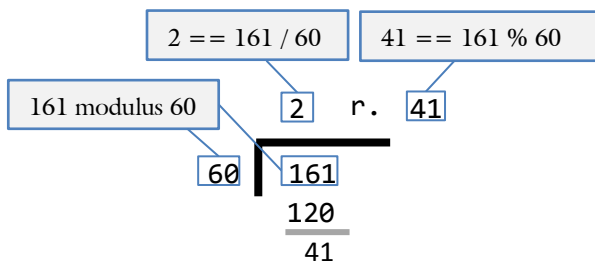
Modulus %

Recall that integer division drops the remainder of the division problem. What if you want to know the remainder? This is the purpose of the modulus operator (%). Consider the following code:

```
{
    int remainder = 19 % 10;
    cout << remainder;    // the output is 9
}
```

In this case, when you divide 19 by 10, the remainder is 9. Therefore, the value of `remainder` will be 9 in this case. For example, consider the following problem:

```
{
    int totalMinutes = 161;           // The movie "Out of Africa" is 161 minutes
    int numHours     = totalMinutes / 60; // The movie is 2 hours long ...
    int numMinutes   = totalMinutes % 60; // ... plus 41 minutes
}
```



Assignment =

In mathematics, the equals symbol $=$ is a statement of equality. You are stating that the right-side and the left-side are the same or balanced. In C++, the equals symbol is a statement of assignment. You are specifying that the evaluation of the right-side is to be assigned to the variable on the left-side. Consider the following code:

```
{
    int x = 2;
    x = x + 1;    // the value of x is updated from 2 to 3. We can
}               // change the value of variables in C++
```

The second statement would not be possible in mathematics; there is no value for x where $x=x+1$ is true. However, in C++, this is quite straightforward: the right-side evaluates to 3 and the variable on the left is assigned to that value. It turns out that adding a value to a variable is quite common. So common, in fact, that a shorthand is offered:

```
{
    int x = 2;
    x += 1;    // the new value of x is 3
}
```

The `+=` operator says, in effect, add the right-side to the variable on the left-side. The end result is the x being updated to the value of 3. The most common variants are:

Operator	Description	Use
<code>+=</code>	Add and assign	Add onto
<code>-=</code>	Subtract and assign	Subtract from
<code>*=</code>	Multiply and assign	Multiply by
<code>/=</code>	Divide and assign	Subdivide

Step 3 - Converting

The final step in evaluating an expression is to convert data from one type to another. This arises from the fact that you can't add an integer to a floating point number. You can add two ints or two floats, but not an int to a float. Consider the following code:

```
cout << 4 + 3.2 << endl;
```

In this example, there are two possibilities: either convert the integer 4 into the float 4.0 or convert the float 3.2 into the integer 3. C++ will always convert ints to floats and bools to ints in these circumstances. It is important to note, however, that this conversion will only happen immediately before the operator is evaluated.

Casting

Rather than allowing the compiler to convert integers or values from one data type to another, it is often useful to perform that conversion yourself explicitly. This can be done with casting. Casting is the process of specifying that a given value is to be treated like another data-type just for the purpose of evaluating a single expression. Consider the following code:

```
{
    int value = 4;
    cout << "float:  " << (float)value << endl;    // the output is "float:  4.0"
    cout << "integer: " << value << endl;         // the output is "integer: 4"
}
```

In this case, the output of the first cout statement will be 4.0 because the integer value 4 will be converted to a floating point value 4.0 in this expression. The value in the variable itself will not be changed; only the evaluation of that variable in that particular expression. The second cout statement will display 4 in this case.

There are a few quirks to casting. First, the variable you are casting does not change. Once you declare a variable as a given data-type, it remains that data-type for the remainder of the program. Casting just changes how that variable behaves for one expression.

Second, not all data-types convert in the most obvious way. Consider converting ints and bools:

```
{
    bool a = (bool)7;           // true    any number but 0 turns into true
    bool b = (bool)0;           // false   only zero turns to false
    int  c = (int>true;         // 1       true always becomes 1
    int  d = (int>false;        // 0       false always becomes 0
}
```

Sam's Corner

There are actually two notations for casting in C++. The older notation, presented above, was inherited from the C programming language and is somewhat deprecated. It still works, but purists will prefer the new notation. The new notation for casting has two main variants: static cast corresponding to casting that happens at compile time, and dynamic cast which happens at runtime. All the casting we do with procedural C++ can be static. We won't need to use dynamic casting until we learn about Object Oriented programming in CS 165.

```
{
    int value = 4;
    cout << "float: " << static_cast<float>(value) << endl;
}
```



Putting it all together

So how does this work together? Consider the following example:

```
{
    int f = 34;
    int c = 5.0 / 9 * (f - 32);
}
```

The most predictable way to evaluate the value of the variable `c` is to handle this one step at a time:

1. `int c = 5.0 / 9 * (f - 32);` // The original statement
2. `int c = 5.0 / 9 * (34 - 32);` // Step 1. Substitute the value `f` for 34
3. `int c = 5.0 / 9 * 2;` // Step 2. Perform subtraction: `2 == 34 - 32`
4. `int c = 5.0 / 9.0 * 2;` // Step 3. Convert 9 to 9.0 for floating point division
5. `int c = 0.555556 * 2;` // Step 2. Perform floating point division: `0.55555 == 5.0 / 9.0`
6. `int c = 0.555556 * 2.0;` // Step 3. Convert 2 to 2.0 for floating point multiplication
7. `int c = 1.111111;` // Step 2. Perform multiplication: `1.11111 == 0.555556 * 2.0`
8. `int c = 1;` // Step 3. Convert 1.111111 to the integer 1 for assignment



Sue's Tips

Seemingly simple expressions can be quite complex and unpredictable when data-type conversion occurs. It is far easier to use only one data-type in an expression. In other words, don't mix floats and ints!

Example 1.3 - Compute Change

Demo

This example will demonstrate how to evaluate simple expressions, how to update the value in a variable, casting, and how to use modulus.

Problem

Write a program to prompt the user for an amount of money. The program will then display the number of dollars, quarters, dimes, nickels, and pennies required to match the amount.

Solution

In this example, the user is prompted for a dollar amount:

```
// prompt the user
cout << "Please enter a positive dollar amount (ex: 4.23): ";
float dollars;
cin >> dollars;
```

Next it is necessary to find the number of cents. This is done by multiplying the dollar variable by 100. Note that dollars have a decimal so they must be in a floating point number. Cents, however, are always whole numbers. Thus we should store it in an integer. This requires conversion through casting.

```
// convert to cents
int cents = (int)(dollars * 100.00);
```

Finally we need to find how many Dollars (and Quarters, Dimes, etc) are to be sent to the user. We accomplish this by performing integer division (where the decimal is removed).

```
cout << "Dollars: " << cents / 100 << endl;
```

After we extract the dollars, how many cents are left? We compute this by finding the remainder after dividing by 100. We can ask for the remainder by using the modulus operator (`cents % 100`). Since we want to assign the new amount back to the cents variable, we have two options:

```
cents = (cents % 100);
```

This is exactly the same as:

```
cents %= 100;
```

Challenge

As a challenge, try to modify the above program so it will not only compute change with coins, but also for bills. For example, it will display the number of \$1's, \$5's, \$10's, and \$20's.

See Also

The complete solution is available at [1-3-computeChange.cpp](#) or:

```
/home/cs124/examples/1-3-computeChange.cpp
```



Problem 1

Please write the variable declaration used for each variable name:

- numberStudents: _____
- pi: _____
- hometown: _____
- priceApples: _____

Please see page 36 for questions

Problem 2

How much space in memory does each variable take?

- bool value; _____
- char value[256]; _____
- char value; _____
- long double value; _____

Please see page 36 for questions

Problem 3

Insert parentheses to indicate the order of operations:

a = a + b * c++ / 4

Please see page 47 for a hint.

Problem 4

What is the value of yard at the end of execution?

```
{  
    float feet = 7;  
    float yards = (1/3) feet;  
}
```

Answer:

yards == _____

Please see page 49 for a hint.

Problem 5

What is the value of a?

```
int a = (2 + 2) / 3;
```

Answer:

a == _____

Please see page 49 for a hint.

Problem 6

What is the value of b?

```
int b = 2 / 3 + 1 / 2;
```

Answer:

b == _____

Please see page 49 for a hint.

Problem 7

What is the value of c?

```
int f = 34;  
int c = 5 / 9 * (f - 32);
```

Answer:

c == _____

Please see page 49 for a hint.

Problem 8

What is the value of d?

```
int d = (float) 1 / 4 * 10;
```

Answer:

d == _____

Please see page 51 for a hint.

Problem 9

Write a program to prompt the user for a number of days, and return the number of days and weeks

Example:

```
How many days: 17
               weeks: 2
               days: 3
```

Please see page 49 for a hint.

Problem 10

What is the output?

```
{
    int dateOfBirth = 1987;
    int currentYear = 2006;

    cout << "age is "
          << currentYear++ - dateOfBirth
          << endl;

    cout << "age is "
          << currentYear++ - dateOfBirth
          << endl;
}
```

Answer:

Please see page 49 for a hint.

Temperature Conversion

Write a program to convert Fahrenheit to Celsius. This program will prompt the user for the Fahrenheit number and convert it to Celsius. The equation is:

$$C = 5/9(F - 32)$$

The program will prompt the user for the temperature, compute the Celsius value, and display the results.

Hint: If you keep getting zero for an answer, you are probably not taking integer division into account. Please review the text for insight as to what is going on.

Hint: If the last test fails, then you are probably not rounding correctly. Note that integers cannot hold the decimal part of a number so they always round down. If you use `precision(0)`, then the rounding will occur the way you expect.

Example

User input is in underline.

```
Please enter Fahrenheit degrees: 72
Celsius: 22
```

Assignment

The test bed is available at:

```
testBed cs124/assign13 assignment13.cpp
```

Don't forget to submit your assignment with the name "Assignment 13" in the header.

Please see page 49 for a hint.